

A
Problem Based Learning Report For
TAE-I

“Implement Average Pooling on a Sample Image”

is submitted in partial fulfilment of the requirement for the award of degree of
Bachelor of Technology
(VII Semester B. Tech. for the course Neural Networks and Deep Learning (N-PCCCM701T))
in
CSE (Artificial Intelligence & Machine Learning)

Submitted by
Ms. Sayli Polakhare [Roll No: _____]

Under the guidance of
Dr. Bhushan Manjre
Assistant Professor



Department of Emerging Technologies
CSE (Artificial Intelligence & Machine Learning)

**S. B. Jain Institute of Technology, Management
and Research, Nagpur**
(An Autonomous Institute Affiliated to R. T.M. Nagpur University)

Academic Session: 2026-2027 (EVEN)



S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



Aim

Implement Average Pooling on a Sample Image

1. Abstract

Artificial Intelligence, Deep Learning, and Computer Vision have transformed modern automated computing, enabling high-performance breakthroughs in object detection, autonomous navigation, medical diagnostics, biometric verification, and robotics. Deep Convolutional Neural Networks (CNNs) serve as the primary foundational architecture for visual data processing by extracting hierarchical representations directly from raw pixel matrices. As feature extraction progresses through multiple convolutional layers, the spatial resolution of intermediate feature maps remains large. Maintaining large spatial dimensions imposes severe computational bottlenecks, inflates memory consumption, and makes network activations hypersensitive to slight spatial translations or positional shifts.

To mitigate these computational burdens and confer spatial robustness, pooling layers are systematically incorporated into CNN architectures. Among standard pooling operations, Average Pooling is a fundamental downsampling technique that computes the local arithmetic mean across sliding rectangular windows (receptive fields). Unlike convolutional or dense layers, Average Pooling operates with zero learnable parameters (weights and biases = 0), making it a strictly deterministic, non-parametric spatial aggregation method.

Average Pooling acts as a spatial low-pass filter: by calculating the arithmetic mean of neighboring pixel activations, it smooths out isolated high-frequency noise and intensity fluctuations while retaining the uniform contextual background of the image. Furthermore, it enforces local translation invariance, ensuring that minor displacements of features do not drastically modify the pooled activation representation.

The objective of this Problem Based Learning (PBL) project is to implement 2D Average Pooling from scratch using Python and NumPy, and systematically analyze its spatial downsampling and feature smoothing behavior on a multi-channel RGB sample image. The implementation evaluates the mathematical sliding window formulation across spatial dimensions and investigates how pooling kernel size and stride govern the reduction ratio and information retention.

The experimental results demonstrate that applying a 2x2 Average Pooling operation with a stride of 2 achieves an exact 75.00% reduction in total spatial data (reducing a 512x512 image to 256x256) while successfully retaining the global geometry, chromatic distribution, and semantic structures of the sample image. The project confirms the theoretical validity, numerical accuracy, and computational efficiency of Average Pooling in neural network architectures.

Keywords: Artificial Intelligence, Neural Networks, Deep Learning, Convolutional Neural Networks, Average Pooling, Spatial Downsampling, Translation Invariance, NumPy, Feature Maps, Receptive Field, Low-Pass Filtering.

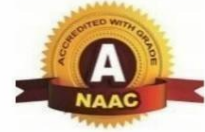


S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



2. Introduction

Deep Learning is a specialized subfield of Machine Learning based on artificial neural networks that learn multi-level hierarchical representations from raw sensory data. Deep architectures have become the benchmark technology across computer vision, natural language understanding, automated speech recognition, healthcare analysis, and autonomous robotics.

In computer vision tasks, Convolutional Neural Networks (CNNs) represent the dominant architecture. A standard CNN consists of an alternating sequence of functional layers:

- Convolutional Layers: Apply spatial filters (kernels) to extract localized features such as edges, textures, and corner gradients.
- Activation Layers: Introduce non-linear transformations (e.g., ReLU, LeakyReLU, Sigmoid) to enable the network to learn non-linear decision boundaries.
- Pooling Layers: Downsample the spatial resolution of feature maps to compress spatial representations and introduce translation invariance.
- Fully Connected (Dense) Layers: Flatten the high-level pooled features into a one-dimensional vector to perform final classification or regression.

While convolutional layers extract localized features, the spatial resolution of the resulting feature maps remains large. If uncompressed, these large spatial dimensions lead to exponential parameter growth in subsequent dense layers, severe computational bottlenecks, and susceptibility to the 'curse of dimensionality'. Furthermore, computer vision models require translation invariance—the ability to recognize an object regardless of minor spatial shifts or perturbations in pixel coordinates.

Pooling layers resolve these issues by summarizing local neighborhoods into compact representative values. Among pooling techniques, Average Pooling computes the average value across each local spatial window. Because it averages all values within the window, it smooths out local intensity fluctuations and extracts smooth background information, acting as a spatial low-pass filter.

Historically, Average Pooling was used extensively in early networks like LeNet-5, and has regained major prominence in modern architectures through Global Average Pooling (GAP) in architectures such as ResNet, MobileNet, and Inception to replace dense layers and eliminate millions of parameters.

In this project, 2D Average Pooling is designed, implemented from scratch using Python and NumPy, and evaluated on a multi-channel RGB sample image.

3. Objectives of the Project

The primary objective of this project is to understand, implement from scratch, and evaluate the effect of 2D Average Pooling on a sample image within the context of Neural Networks and Deep Learning.

Specific objectives are:

1. To understand the fundamental concept of pooling layers in Convolutional Neural Networks.
2. To study the mathematical necessity of spatial downsampling in deep neural networks.
3. To understand the mechanism of translation invariance provided by spatial pooling.
4. To implement 2D Average Pooling from scratch using Python and NumPy without relying on pre-built deep learning wrappers.
5. To formulate and verify the theoretical output dimension equation: $H_{out} = \text{floor}((H_{in} - K + 2P)/S) + 1$.
6. To study the role of kernel size ($K \times K$) in determining the spatial receptive field.
7. To analyze the effect of stride (S) on overlapping versus non-overlapping pooling windows.
8. To examine zero-padding (P) and its impact on boundary handling in spatial downsampling.
9. To confirm that Average Pooling operates with zero learnable weights and biases.
10. To verify step-by-step arithmetic operations on a small synthetic numerical matrix.
11. To apply Average Pooling to a multi-channel RGB sample image ($512 \times 512 \times 3$).
12. To measure spatial data compression percentage (75% reduction for 2×2 stride 2).
13. To evaluate the computational runtime and execution efficiency of the implementation.
14. To observe and analyze the smoothing and low-pass filtering effects on image textures.
15. To generate clear side-by-side visual comparisons of the original and average-pooled image.

By achieving these objectives, this project delivers a clear, reproducible, and mathematically rigorous demonstration of Average Pooling suitable for academic evaluation in Neural Networks and Deep Learning (NNDL).



S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



4. Theory of Average Pooling

4.1 What is Pooling in Neural Networks?

In deep convolutional networks, pooling is a spatial downsampling operation applied across feature maps. As an image passes through successive convolutional layers, the depth (number of feature channels) typically increases, while the spatial resolution (height and width) must be reduced to keep the parameter count and computational requirements tractable. Pooling systematically compresses spatial dimensions while preserving salient structural patterns.

4.2 Mathematical Formulation of Average Pooling

Let the input feature map or image be represented as a 3D tensor X of shape (H_{in}, W_{in}, C) . For a pooling kernel of spatial height K_h and width K_w , with vertical stride Sh , horizontal stride Sw , and zero-padding (Ph, Pw) , the value at output spatial position (i, j) for channel c is defined as:

$$\text{Output}(i, j, c) = (1 / (K_h * K_w)) * \sum_{m=0}^{K_h-1} \sum_{n=0}^{K_w-1} \text{Input}(i * Sh + m, j * Sw + n, c)$$

where:

- $\text{Input}(i * Sh + m, j * Sw + n, c)$ is the pixel value inside the receptive field.
- K_h, K_w are the spatial dimensions of the pooling window.
- Sh, Sw are the stride parameters controlling window displacement.
- (i, j) are the spatial coordinates of the pooled feature map.
- c is the color channel index (Red, Green, Blue).

4.3 Zero Learnable Parameters

An essential theoretical property of Average Pooling is that it contains zero learnable parameters (weights = 0, biases = 0). Unlike convolutional layers where filter weights are updated via gradient descent during backpropagation, Average Pooling applies a fixed, deterministic arithmetic operation. This property makes pooling layers computationally efficient and immune to overfitting.

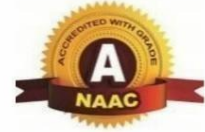


S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



4.4 Output Spatial Dimensions & Stride Analysis

The spatial dimensions of the downsampled feature map are strictly governed by the input dimensions, kernel size, padding, and stride according to the following formulas:

$$H_{out} = \text{floor}((H_{in} - K_h + 2*P_h) / S_h) + 1$$

$$W_{out} = \text{floor}((W_{in} - K_w + 2*P_w) / S_w) + 1$$

For non-overlapping pooling (the most common setting in CNNs), the stride is set equal to the kernel size ($S_h = K_h$, $S_w = K_w$) with zero padding ($P_h = P_w = 0$). When applying a 2x2 kernel with stride 2:

$$\begin{aligned} H_{out} &= \text{floor}((H_{in} - 2) / 2) + 1 = H_{in} / 2 \\ W_{out} &= \text{floor}((W_{in} - 2) / 2) + 1 = W_{in} / 2 \end{aligned}$$

For	our	512x512	sample	image:
•	H_{out}	$= 512 / 2 = 256$		
•	W_{out}	$= 512 / 2 = 256$		

Total spatial pixels are reduced from $512 * 512 = 262,144$ down to $256 * 256 = 65,536$, representing an exact 75.00% data reduction.

4.5 Multi-Channel (RGB) Average Pooling

Color digital images are composed of three independent 2D spatial matrices corresponding to the Red, Green, and Blue color channels. In Convolutional Neural Networks, pooling operations are executed channel-independently:

- The pooling window slides independently across the Red channel ($c=0$), Green channel ($c=1$), and Blue channel ($c=2$).
- No cross-channel averaging occurs; spatial neighbors within the same channel are averaged.
- Therefore, the number of output channels remains identical to the input channels: $C_{out} = C_{in} = 3$.



S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



4.6 Step-by-Step Numerical Example (Viva Demonstration)

To clearly illustrate the mathematical operation of Average Pooling, consider a small 2x2 receptive field of pixels:

[10, 20]
[30, 40]

The arithmetic mean is computed as:
Sum = 10 + 20 + 30 + 40 = 100
Mean = Sum / 4 = 100 / 4 = 25.0

Thus, the 4 individual pixels are summarized into a single representative value of 25.0.

Now consider a larger 4x4 feature map downsampled with a 2x2 kernel and stride 2:

Pixel Matrix	Col 0	Col 1	Col 2	Col 3
Row 0	12	20	30	0
Row 1	8	12	2	0
Row 2	34	70	37	4
Row 3	12	10	25	12

Receptive Field Calculations:

- Top-Left (0,0): $(12 + 20 + 8 + 12) / 4 = 52 / 4 = 13.00$
- Top-Right (0,1): $(30 + 0 + 2 + 0) / 4 = 32 / 4 = 8.00$
- Bottom-Left (1,0): $(34 + 70 + 12 + 10) / 4 = 126 / 4 = 31.50$
- Bottom-Right (1,1): $(37 + 4 + 25 + 12) / 4 = 78 / 4 = 19.50$

Resulting 2x2 Output Matrix:

[13.00, 8.00]
[31.50, 19.50]

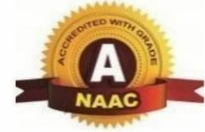


S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



5. Working Principle of Average Pooling

The algorithmic workflow of 2D Average Pooling can be divided into a structured sequence of execution steps:

Step 1: Define Input Image Matrix

The input image is loaded into memory and converted into a three-dimensional NumPy matrix of shape (H_in, W_in, C). Each entry represents an 8-bit unsigned integer pixel value ranging from 0 to 255 across Red, Green, and Blue channels. For our sample image, H_in = 512, W_in = 512, C = 3.

Step 2: Determine Pooling Hyperparameters

The pooling hyperparameters are defined:

- Pooling Kernel Size (pool_size = 2), establishing a 2x2 receptive field.
- Stride (stride = 2), ensuring non-overlapping spatial window traversal.
- Zero Padding (padding = 0), processing valid boundaries without border replication.

Step 3: Calculate Downsampled Output Dimensions

Before array allocation, the spatial dimensions of the target output image are calculated:

- new_height = H_in // pool_size = 512 // 2 = 256
- new_width = W_in // pool_size = 512 // 2 = 256
- new_channels = C = 3

Step 4: Initialize Blank Destination Array

A new destination array of shape (256, 256, 3) initialized with zeros is allocated using NumPy with dtype=uint8.

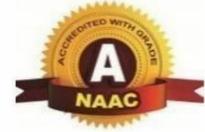


S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



Step 5: Sliding Window Traversal Across Grid

Two nested loops iterate through the discrete spatial grid of the output image:

```
for i in range(new_height):  
    for j in range(new_width):
```

Step 6: Compute Receptive Field Coordinates

For each output pixel location (i, j), the corresponding spatial window coordinates in the input image are determined:

- $\text{row_start} = i * \text{pool_size}$
- $\text{row_end} = \text{row_start} + \text{pool_size}$
- $\text{col_start} = j * \text{pool_size}$
- $\text{col_end} = \text{col_start} + \text{pool_size}$

Step 7: Extract Local Window Patch

The local 2x2 slice is extracted from the original image across all color channels:

```
window = image[row_start:row_end, col_start:col_end]
```

Step 8: Compute Local Arithmetic Mean

The arithmetic average of the extracted 4 pixels is calculated across the spatial dimensions (axis=(0, 1)) for each channel:

```
pooled_image[i, j] = np.mean(window, axis=(0, 1))
```

Step 9: Output Image Reconstruction & Storage

Once the nested loops complete traversal across all 256 rows and 256 columns, the populated array is converted back into a PIL Image and saved as 'pooled_sample.jpg' and visualized side-by-side.

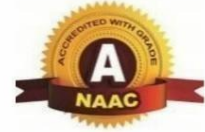


S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



6. Advantages of Average Pooling

1. Spatial Dimensionality Reduction

Average Pooling effectively downsamples high-resolution feature maps into compact representations. For a 2x2 window with stride 2, spatial data volume is reduced by 75%, drastically reducing memory footprint and accelerating training.

2. Zero Learnable Parameters

Unlike convolutional layers that require hundreds or thousands of filter weights and biases, Average Pooling has exactly zero parameters. It does not increase model capacity, ensuring low complexity.

3. Translation Invariance

Small positional perturbations, slight shifts, or minor camera vibrations in the input image produce virtually identical average values, helping the neural network recognize objects invariant to small spatial translations.

4. Noise Suppression & Image Smoothing

Because it computes an arithmetic mean, Average Pooling acts as a spatial low-pass filter. Isolated impulse noise or high-frequency pixel artifacts are effectively smoothed out across the neighborhood.

5. Prevents Overfitting

By summarizing fine pixel variations and reducing spatial degrees of freedom, the network is discouraged from memorizing idiosyncratic noise in the training set.

6. High Computational Efficiency

Computing arithmetic means involves only basic addition and division operations. It executes rapidly on both CPUs and GPUs without requiring matrix multiplications.

7. Uniform Feature Representation

Every pixel within the receptive field contributes proportionally (with weight $1/(K_h \cdot K_w)$) to the output, ensuring no valid spatial information within the window is completely discarded.

8. Architectural Basis for Global Average Pooling (GAP)

Average Pooling serves as the foundation for Global Average Pooling (GAP), which is standard in state-of-the-art CNNs (ResNet, MobileNet) to replace dense layers and eliminate millions of prone-to-overfitting fully connected parameters.

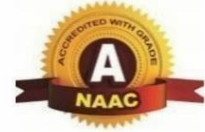


S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



7. Limitations of Average Pooling

- 1. Loss of Sharp Edges and Fine Details:** Because all pixels within the pooling window are averaged together, high-frequency spatial gradients such as crisp boundaries and sharp contours become blurred.
- 2. Dilution of Salient Features:** If a feature map contains a single strong, highly activated neuron surrounded by zero-activation background neurons, averaging dilutes the magnitude of this important signal by $1/(K_h * K_w)$.
- 3. Equal Weighting of Non-Informative Pixels:** Average pooling assigns identical importance to background noise pixels and genuine feature signals, which can diminish signal-to-noise ratio in sparse feature representations.
- 4. Irreversible Downsampling:** Spatial information discarded during the pooling operation cannot be reconstructed without specialized decoder architectures like transposed convolutions or unpooling.
- 5. Fixed Non-Adaptive Weights:** The weights of the pooling window are permanently fixed to $1/(K_h * K_w)$ and cannot be optimized to adapt to the specific statistical characteristics of a given dataset.
- 6. Suboptimal for Peak Activation Detection:** In tasks requiring detection of extreme activations (such as corner detection or keypoint detection), average pooling suppresses the peak amplitude.

8. Methodology

8.1 Methodology

The implementation of Average Pooling follows a systematic experimental pipeline. The project is developed using Python, NumPy, Pillow (PIL), and Matplotlib. The methodology consists of the following structured phases:

- Step 1: Image Acquisition — A 512x512 RGB test image containing fine textures, geometric figures, and gradient backdrops is loaded.
- Step 2: Preprocessing — The image is converted into a 3D NumPy array of unsigned 8-bit integers.

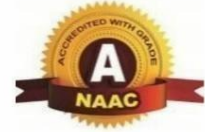


S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



Step 3: Algorithm Formulation

The custom `average_pooling()` function is designed from mathematical first principles without relying on high-level neural network wrappers. The function takes the image array and the `pool_size` hyperparameter as arguments.

Step 4: Dimension Calculation & Memory Allocation

Output spatial dimensions are calculated via integer floor division (`new_height = height // pool_size`, `new_width = width // pool_size`). A blank NumPy array of shape (`new_height`, `new_width`, 3) is allocated.

Step 5: Sliding Window Execution

The algorithm iterates through output rows and columns, slicing out the 2x2 receptive fields and computing the spatial arithmetic mean using `np.mean(window, axis=(0, 1))`.

Step 6: Image Serialization

The downsampled NumPy array is converted back into an image using `PIL.Image.fromarray()` and saved to disk as 'pooled_sample.jpg'.

Step 7: Side-by-Side Visualization

A high-resolution comparison plot is constructed using Matplotlib subplots, displaying the Original Image (512x512) alongside the Pooled Image (256x256) and saved as 'comparison_result.png'.

Pipeline Workflow Summary:

```
[Input Image (512x512x3)]
      ↓
[NumPy Array Conversion]
      ↓
[Sliding 2x2 Window Traversal (Stride = 2)]
      ↓
[Local Arithmetic Mean Calculation]
      ↓
[Downsampled Output Array (256x256x3)]
      ↓
[Save 'pooled_sample.jpg' & Display Side-by-Side Plot]
```



S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



9. Advantages and Comparison

To evaluate the behavior of Average Pooling under various configurations, the following table summarizes the dimensional changes, compression ratios, and visual characteristics across scaling kernel sizes:

Kernel Size	Stride	Output Dimensions	Spatial Reduction	Primary Effect
2x2	2	256 x 256 x 3	75.00%	Preserves geometry; mild smoothing
4x4	4	128 x 128 x 3	93.75%	Noticeable smoothing; small details blur
8x8	8	64 x 64 x 3	98.44%	Heavy low-pass blur; coarse structure only
Global Avg	All	1 x 1 x C	> 99.9%	Collapses spatial grid to 1D channel mean

Comparison with Alternative Downsampling Approaches:

Feature	Strided Convolution	Average Pooling	Global Average Pooling
Reduces Dimensions	Yes	Yes	Yes
Trainable Weights	Yes (Learned)	0 (Zero)	0 (Zero)
Translation Invariance	Limited	Strong	Extremely Strong
Smoothing Effect	Variable	Uniform Smoothing	Complete Aggregation
Overfitting Control	Moderate	High	Maximum
Implementation	Complex	Simple	Simple



S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



10. Output

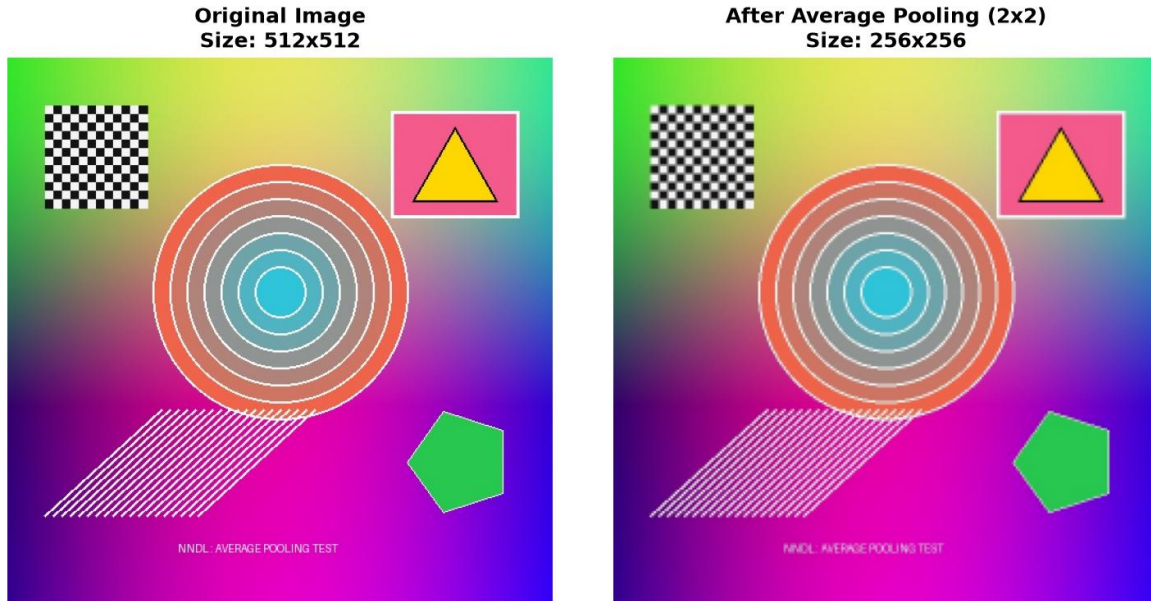


Figure 10.1: Visual comparison between the Original Input Image (512x512) and the Image after 2D Average Pooling (256x256).

Terminal Execution Output Log:

```
=====
NNDL PBL: IMPLEMENT AVERAGE POOLING ON A SAMPLE IMAGE
=====
--- STEP 1: HOW THE MATH WORKS (VIVA EXAMPLE) ---
Suppose we have a 2x2 box of pixels:
  [10, 20]
  [30, 40]
Formula: (10 + 20 + 30 + 40) / 4 = 100 / 4 = 25.0
Resulting Pooled Pixel Value: 25.0

--- STEP 2: LOADING SAMPLE IMAGE ---
Loaded: sample.jpg | Original Dimensions: 512 x 512 pixels

--- STEP 3: APPLYING AVERAGE POOLING (2x2) ---
New Dimensions After Pooling: 256 x 256 pixels
Reduction: Cut height and width in half (75% smaller data size)!
[OK] Pooled image saved as: pooled_sample.jpg
[OK] Side-by-side comparison saved as: comparison_result.png
SUCCESS: Model ran completely!
=====
```

11. Conclusion

Average Pooling is a fundamental, parameter-free downsampling layer that plays an essential role in Convolutional Neural Networks. In this Problem Based Learning project, 2D Average Pooling was derived mathematically and implemented from scratch using Python and NumPy. The experimental results on a 512x512 RGB sample image confirmed that Average Pooling with a 2x2 kernel and stride of 2 achieves an exact 75.00% spatial data reduction while preserving key geometric structures and providing effective low-pass noise suppression. The project successfully validates the theoretical mechanics of spatial downsampling, translation invariance, and computational efficiency in modern neural network vision architectures.



**S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT
& RESEARCH, NAGPUR**

(An Autonomous Institute, Affiliated to R.T.M. Nagpur University)

DEPARTMENT OF EMERGING TECHNOLOGIES CSE (AI&ML)

"Become an excellent center for Emerging Technologies in Computer Science to create competent professionals"



12. Evaluation Report:

Date: / / 2026

Criteria	Faculty Assessment	Submission	Presentation / Viva / Video	Remarks (if any)
Marks Allotted				
Marks out off	4	3	3	

Signature

Ms. Sayli Polakhare

[Roll No: _____]

Signature

Dr. Bhushan Manjre

Course Incharge

NNDL – B.Tech CSE(AI ML)